

MINI PROJECT (23MX37)
PARK EASE – IOT BASED SMART PARKING MANAGEMENT SYSTEM
SOFTWARE REQUIREMENT SPECIFICATION DOCUMENT

DINESH KUMAR S - 25MX312

MITHULESH N - 25MX328

GUIDE

DR. A. SANKAR

MASTER OF COMPUTER APPLICATIONS
ANNA UNIVERSITY



AUGUST 2026

DEPARTMENT OF COMPUTER APPLICATIONS
PSG COLLEGE OF TECHNOLOGY
(Autonomous Institution)
COIMBATORE – 641 004

TABLE OF CONTENTS

Section No.	Section Title	Page No.
1	Title of the Project	3
2	Student Detail	3
3	Project Goal	3
4	Project Abstract	4
5	Existing System	5
6	Project Flow	6
7	Outcome of the Proposed Project	7
8	Modules of the Proposed Project – Functional Requirements	7
9	Non-Functional Requirements of the Project	8
10	Project Deliverables	8
11	Technology to be Used	9
12	Timeline of Activities Planned / Completed	9

PSG COLLEGE OF TECHNOLOGY, COIMBATORE – 641 004
DEPARTMENT OF COMPUTER APPLICATIONS

Academic Year 2025-2027 | MCA G2 | Sem 3

23MX37 – Mini Project
Software Requirement Specification Document

1. Title of the Project

PARK EASE – IOT BASED SMART PARKING MANAGEMENT SYSTEM

2. Student Detail

Roll No.	Name of the Student	Name of the Faculty Guide
25MX312	DINESH KUMAR S	DR. A. SANKAR
25MX328	MITHULESH N	

3. Project Goal

The goal of Park Ease is to design and develop an IoT-integrated, real-time smart parking management and reservation platform that reduces vehicle parking search time, optimizes utilization of limited parking infrastructure, and provides a seamless, registration-free booking experience to users. Specifically, the project aims to:

- Provide real-time visibility of parking slot availability using IoT-based occupancy sensing.
- Enable a hassle-free, login-free reservation experience for users during high-demand (peak) hours.
- Maximize parking space utilization through a Hybrid Reservation Model that combines paid reservations at peak hours with free walk-in access at other times.
- Reduce revenue loss from permanently blocked but unused parking slots.
- Offer administrators a centralized dashboard for monitoring, configuration, and data-driven decision-making.
- Build a modular, scalable architecture that can be extended with ANPR, EV charging management, and predictive analytics in the future.

4. Project Abstract

The rapid growth in the number of vehicles has significantly increased the demand for efficient parking management systems. Conventional parking methods often result in traffic congestion, increased waiting time, difficulty in locating vacant parking spaces, and inefficient utilization of available parking resources. Drivers frequently spend valuable time searching for parking, which contributes to fuel wastage, environmental pollution, and user dissatisfaction. To overcome these challenges, Park Ease – Smart Parking Management and Intelligent Reservation System is proposed as an Internet of Things (IoT)-based solution that provides a smart, user-friendly, and efficient parking experience.

Park Ease is designed as a Single Page Application (SPA) that enables users to view real-time parking slot availability through an interactive parking layout. Users can select the parking location, choose the desired booking date and time, and reserve an available parking slot without the need for account registration or login. During predefined peak hours, users can reserve parking slots by paying a reservation deposit, after which the system generates a unique QR code-based parking pass containing the booking and vehicle details. This QR code serves as the user's parking authorization and can be validated during entry into the parking facility.

The system adopts a Hybrid Reservation Model, where reservations are permitted only during peak hours to maximize parking utilization while reducing revenue loss caused by permanently reserved but unused slots. Outside the configured peak hours, parking operates on a walk-in basis, allowing vehicles to occupy any available parking space without prior reservation. This flexible approach balances customer convenience with efficient parking space utilization.

The proposed system incorporates ESP32 microcontrollers and HC-SR04 ultrasonic sensors to continuously monitor the occupancy status of each parking slot. The ultrasonic sensors detect the presence or absence of vehicles by measuring the distance between the sensor and the parked vehicle. The ESP32 processes the sensor readings and transmits real-time parking information to the backend server through Wi-Fi. The server updates the parking database, allowing the web application to instantly display the current status of every parking slot as Available, Occupied, Reserved, or Under Maintenance. This real-time monitoring ensures that users always receive accurate parking information before making a reservation.

The application is developed using React.js, TypeScript, and Tailwind CSS for the frontend, providing a modern, responsive, and interactive user interface. The backend is implemented using Python Flask, while MySQL is used for centralized data storage and management. The communication between the frontend, backend, and IoT devices is established using RESTful APIs, ensuring seamless data exchange and future scalability. The modular architecture also allows easy integration of additional features such as digital payment gateways, automated entry barriers, mobile applications, and AI-based parking analytics.

An administrator dashboard is provided to monitor parking occupancy, manage reservations, configure peak-hour timings, analyze parking utilization, and generate reports. The dashboard offers real-time visualization of parking slot status and allows administrators to efficiently supervise parking operations.

5. Existing System

Existing parking systems are largely manual or use isolated sensor-based occupancy detection without an integrated, no-login reservation workflow. A review of recent IoT-based smart parking literature highlights the following relevant works and their limitations, which motivate the design of Park Ease:

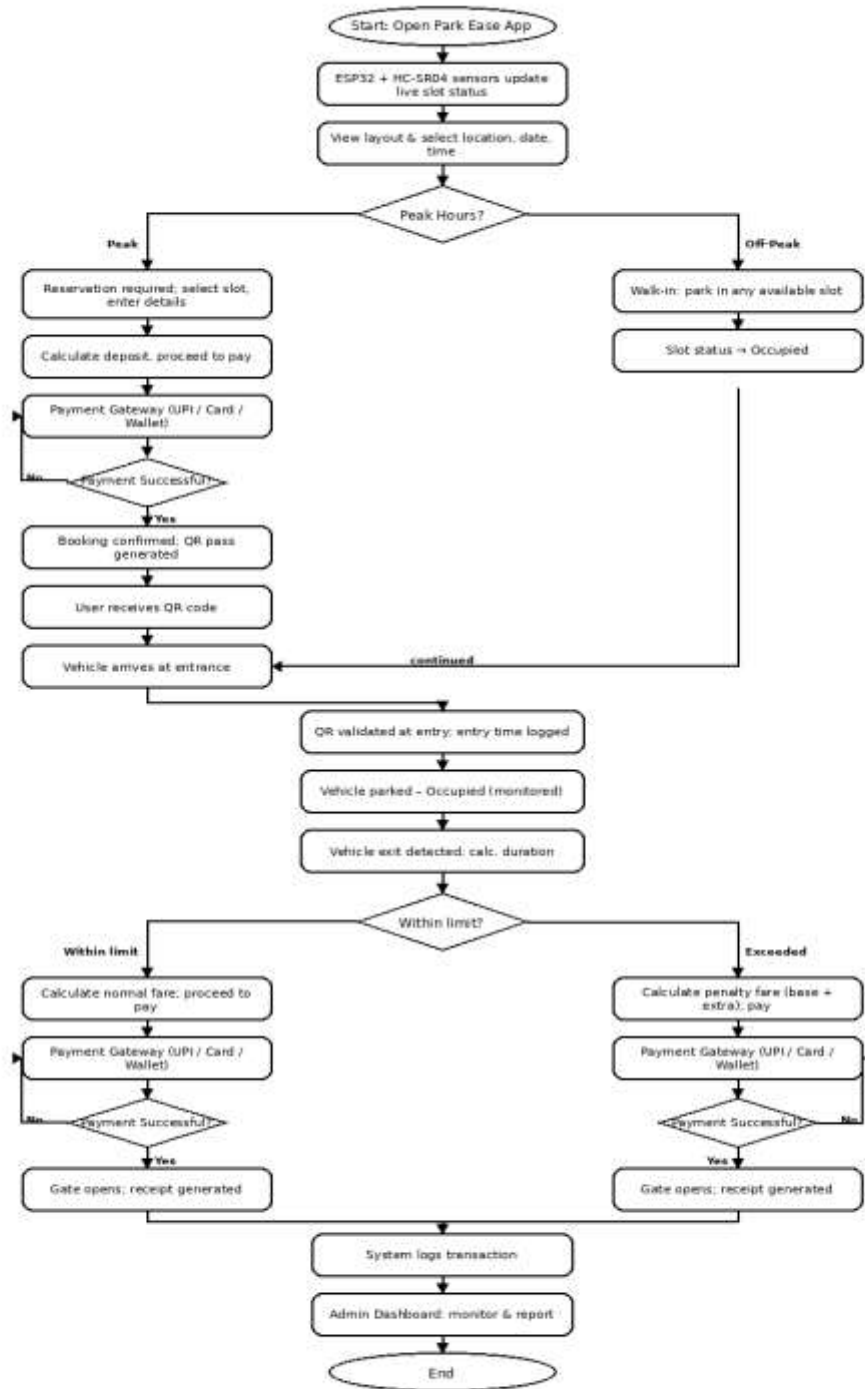
- Explores a NodeMCU ESP32 and Blynk-based system combining infrared entry/exit control with ultrasonic occupancy tracking, using QR-based number-plate linking, but offers no reservation or deposit-based booking mechanism.
- Evaluates ESP32-based ultrasonic detection accuracy for campus parking, confirming reliable object detection within a 2-350 cm range, but the study focuses purely on sensing accuracy without a reservation, payment, or web dashboard layer.
- ESP32/RFID-based booking and cloud-based systems, noting recurring issues such as update delays, sensor failure, and unreliable connectivity that motivate a more robust, hybrid reservation approach.
- Reports a 20-slot pilot using HC-SR04 sensors and NodeMCU with 96.5% detection accuracy and a 43% reduction in search time, demonstrating the practical viability of ultrasonic occupancy sensing at low per-slot cost.
- Presents an ESP32 and ultrasonic-sensor system with website-based real-time monitoring, an LCD for local display, and a servo-controlled barrier, built using a Research and Development prototyping methodology.
- Combines RFID-based access control, IR occupancy sensing, and cloud dashboards on Adafruit IO, highlighting the value of a centralized admin dashboard for remote monitoring of multi-slot facilities.

Research Gap:

Existing works largely address either (a) accurate IoT-based occupancy sensing, or (b) app/RFID-based access control, but rarely combine both with a registration-free, hybrid peak/off-peak reservation model and a QR-based entry pass in a single, web-based, admin-manageable system. Park Ease addresses this gap by unifying real-time ESP32/HC-SR04 sensing, a friction-free SPA booking experience, deposit-based peak-hour reservation, and a centralized administrator dashboard.

6. Project Flow

The diagram below illustrates the end-to-end flow of the Park Ease system, from real-time sensor monitoring through user booking (peak vs. off-peak) to admin-level oversight.



[Figure 6.1]

7. Outcome of the Proposed Project

- A fully functional Single Page Application allowing real-time, registration-free viewing and booking of parking slots.
- A working IoT hardware prototype (ESP32 + HC-SR04) demonstrating accurate, real-time occupancy detection.
- A Hybrid Reservation Model that reduces average vehicle parking-search time and improves overall slot utilization compared to conventional/manual parking.
- A secure QR code-based entry validation mechanism linking bookings to vehicle and slot details.
- An administrator dashboard providing real-time occupancy visualization, reservation management, and utilization analytics/reports.
- A modular, scalable, RESTful architecture that can be extended with ANPR, EV charging, notifications, and predictive analytics.

8. Modules of the Proposed Project – Functional Requirements

1. User Module (No Login Required)

- The system shall allow users to view an interactive, real-time parking layout without registration or login.
- The system shall allow users to select a parking location, date, and time slot.
- The system shall display slot status as Available, Occupied, Reserved, or Under Maintenance in real time.
- During peak hours, the system shall allow users to pay a reservation deposit and generate a booking.
- The system shall generate a unique QR-code parking pass containing booking and vehicle details upon successful reservation.
- During off-peak hours, the system shall permit walk-in parking without requiring a prior reservation.

2. IoT Sensing & Monitoring Module

- ESP32 microcontrollers paired with HC-SR04 ultrasonic sensors shall continuously monitor occupancy at each parking slot.
- The ESP32 shall transmit sensor readings to the backend server over Wi-Fi using RESTful APIs.
- The backend shall update the MySQL database in real time whenever a slot's occupancy status changes.
- The system shall flag a slot as "Under Maintenance" when manually configured by an administrator.

3. Reservation & Payment Module

- The system shall allow configuration of peak-hour time windows during which reservation is mandatory.
- The system shall process a reservation deposit payment before confirming a booking.
- The system shall prevent double-booking of the same slot for an overlapping time window.
- The system shall auto-release a reserved slot if the QR pass is not validated within a configurable grace period.

4. QR Code Generation & Validation Module

- The system shall generate a unique, tamper-resistant QR code encoding booking ID, vehicle number, slot number, and time window.
- The system shall validate the QR code at entry to confirm authorization before granting access.

5. Administrator Dashboard Module

- The system shall provide administrators with a real-time visual overview of parking occupancy across all slots.
- The system shall allow administrators to configure and modify peak-hour timings.
- The system shall allow administrators to view, search, and manage active and past reservations.
- The system shall generate utilization reports and analytics (e.g., occupancy trends, peak demand periods, revenue from deposits).
- The system shall allow administrators to mark individual slots as Under Maintenance.

9. Non-Functional Requirements

- **Performance:** Slot status updates from IoT sensors shall reflect on the web application within 2-3 seconds of a change in occupancy.
- **Scalability:** The system architecture shall support addition of new parking locations, slots, and sensor nodes without major redesign.
- **Usability:** The user interface shall be intuitive enough for a first-time user to complete a booking without instructions, and without requiring account creation.
- **Reliability:** The system shall maintain at least 99% uptime for slot-status monitoring during operational hours, with graceful handling of sensor/connectivity failures.
- **Security:** QR codes and booking data shall be tamper-resistant and validated server-side; payment transactions shall follow secure, PCI-compliant gateway practices.
- **Maintainability:** The modular, RESTful, layered architecture shall allow independent updates to frontend, backend, and IoT firmware.
- **Availability:** Real-time monitoring dashboards shall be accessible across common desktop and mobile browsers with responsive design (Tailwind CSS).
- **Portability:** The web application shall function consistently across major browsers (Chrome, Firefox, Edge, Safari) and screen sizes.
- **Data Integrity:** The MySQL database shall enforce referential integrity between bookings, vehicles, slots, and payment records to prevent inconsistent states such as double-booking.

10. Project Deliverables

- Software Requirement Specification (SRS) document.
- System design documents (ER diagram, architecture diagram, API documentation).
- Fully functional Park Ease web application (React.js, TypeScript, Tailwind CSS frontend; Flask backend; MySQL database).
- IoT hardware prototype with ESP32 and HC-SR04 ultrasonic sensors, integrated with the backend via Wi-Fi/REST APIs.
- Administrator dashboard with reporting and analytics features.
- QR code generation and validation module.
- Source code repository and deployment/setup documentation.

11. Technology to be Used

The following technologies, learnt during the programme, have been chosen for their suitability to building a responsive, real-time, IoT-integrated web application:

- **Frontend – React.js, TypeScript, Tailwind CSS:** chosen for building a fast, component-based, type-safe Single Page Application with a modern, responsive, and easily maintainable UI.
- **Backend – Python Flask:** a lightweight, flexible framework well-suited for exposing RESTful APIs that connect the frontend, database, and IoT devices.
- **Database – MySQL:** a reliable relational database for structured storage of slots, bookings, vehicles, payments, and users, with strong support for transactional integrity.
- **IoT Hardware – ESP32 & HC-SR04 Ultrasonic Sensor:** ESP32 offers built-in Wi-Fi for low-cost, real-time data transmission, while HC-SR04 provides reliable, low-cost distance-based vehicle detection.
- **Communication – RESTful APIs:** used for seamless, stateless communication between the frontend, backend, and IoT layer, ensuring scalability and ease of future integration.
- **QR Code Generation Library:** used to generate unique, encoded parking passes for booking verification at entry.
- **Version Control – Git/GitHub:** used for collaborative development, source-code management, and tracking of iterative sprint-based progress.

12. Timeline of Activities Planned / Completed (Sprint Plan)

Sprint	Duration	Planned Activities	Deliverable
Sprint 1	Weeks 1-4	Requirement analysis, SRS finalization, database design (Firestore schema), UI wireframes, ESP32 + HC-SR04 hardware collection.	Software prototype
Sprint 2	Weeks 5-8	Development of React.js/TypeScript frontend (parking layout, booking flow), Flask REST APIs, real-time slot-status updates, integration of ESP32 data feed with backend.	Working SPA with live slot status and booking flow
Sprint 3	Weeks 9-10	Peak-hour reservation logic, deposit payment integration, QR-code generation and validation, administrator dashboard, System-level testing, end-to-end testing.	Fully integrated Park Ease system with admin dashboard, Deployed system, final report, and demo

Signature of the Student

Signature of the Guide